

Approximation and Optimization on Graphs: the Vertex Cover Problem

Julian OEHLER

Project #3

1 Problem & Goals

This project implements algorithms that solve the vertex cover problem on a graph. The Vertex Cover problem is an NP-hard optimization problem whose objective is to find the subset of vertices $S \subseteq V$ of minimum cardinality such that every edge (u, v) is covered, that is, $\forall (u, v) \in E$, either $u \in S$ or $v \in S$.

To solve this problem, we will implement and compare three algorithms: a greedy algorithm, an LP-relaxation rounding algorithm and a branch-and-bound algorithm.

2 Approach and Design Choices

The project is divided into three parts:

- Cover computing
- Algorithm Benchmarks
- Branch-and-bound scalability

These three parts use the implemented algorithms described below.

2.1 Greedy Vertex Cover Algorithm

The greedy algorithm repeatedly selects an uncovered edge, adds both of its endpoints to the vertex cover and removes all edges that have one of these endpoints from the uncovered edges until all edges are covered.

Theorem 1. *The greedy vertex cover algorithm is a 2-approximation algorithm for the Minimum Vertex Cover problem.*

Proof. Let M be the set of edges selected by the algorithm. We have that no two edges in that set share an endpoint, since after each edge (u, v) was added to the set, all edges having either u or v were removed from the uncovered edges.

Let S be the vertex cover returned by the algorithm. Since two vertices are added to the cover at each iteration, i.e., each time a new edge is selected, it follows that $|S| = 2|M|$ (1).

Let S^* be a minimum vertex cover. By definition, we have that $\forall (u, v) \in E, u \in S^*$ or $v \in S^*$. Since $M \subseteq E$, this same condition must hold for any edge of M . Given that no two edges of M share an endpoint, S^* has at least as much elements as the number of selected edges. Thus, $|S^*| \geq |M|$ (2).

Using (1) and (2), we get that:

$$|S| = 2|M| \leq 2|S^*|$$

That is, the vertex cover returned by the algorithm has cardinality at most twice the one of the minimum vertex cover. Thus, the greedy algorithm is a 2-approximation algorithm. $\square$

2.2 LP-Relaxation Rounding Algorithm

The LP-relaxation rounding algorithm solves a relaxed linear program of the minimum vertex cover problem.

A linear program is an optimization problem where we minimize a linear objective subject to linear constraints. In the relaxed version, each vertex $v \in V$ is associated with a variable $x_v \in [0, 1]$ representing whether the vertex is selected in the cover, where values closer to 1 indicate a stronger selection and values closer to 0 indicate not being selected.

Moreover, in our case, the constraints ensure that for every edge $(u, v) \in E$, at least one of its endpoints is selected, i.e., $x_u + x_v \geq 1$.

The LP is solved using the PuLP library, and the final cover is obtained by selecting all vertices with value at least 0.5. This produces a 2-approximation algorithm.

2.3 Branch-and-Bound Algorithm

Unlike the previous approximation algorithms, the Branch-and-Bound approach computes an exact minimum vertex cover. Starting from an uncovered edge (u, v) , the algorithm recursively branches by adding either u or v to the current cover. Partial solutions whose cardinality is greater than the best solution found so far are not considered, reducing the search space. Although this pruning significantly improves performance, the algorithm remains exponential in the worst case, and therefore does not scale well to large graphs.

3 Results

3.1 Cover computing User Interface

Figure 1 below is an example of what the program returns when asked to compute the cover of the graph defined by the following edges: ("A", "B"), ("B", "C"), ("B", "D"), ("A", "D"), ("D", "E").

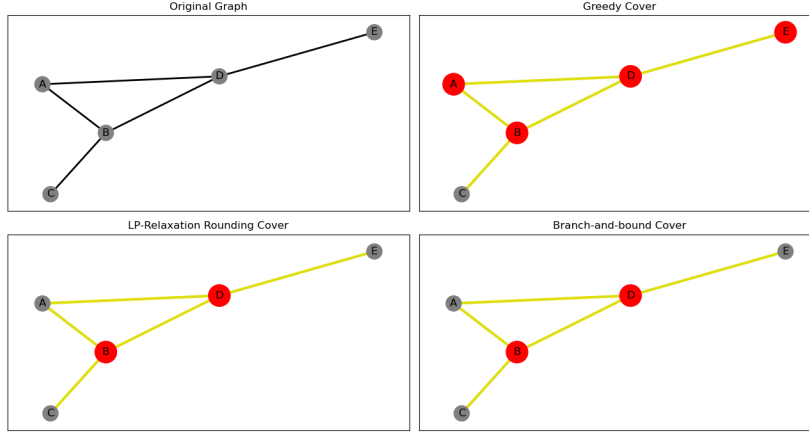


Figure 1: Cover visualizing UI

3.2 Algorithm Benchmarks Results

Finally, the results in Table 1 summarize the findings of the project. They were computed using the set of values $V = \{2, 7, 15, 20, 25, 30\}$ for n , and a fixed graph density of 0.3. The values of n are deliberately low, to be able to compute the exact branch-and-bound cover for each graph.

Algorithm	Mean Emp. Ratio	Worst Emp. Ratio	Max Solved n
Greedy	1.33	2	—
LP-Relax. Rounding	1.45	2	—
Branch-and-Bound	0.94	1.4	50

Table 1: Final Results, Algorithms Against Mean and Worst Empirical Ratio

There are three interpretations that can be made from the table. First, the worst ratio both for greedy and LP-relaxation rounding algorithms is 2. It emphasizes on the fact that these algorithms are 2-approximations of the optimal solution in the worst case.

Moreover, the LP-relaxation rounding algorithm obtained a higher mean empirical ratio than the greedy algorithm on the tested values of n , despite both algorithms being 2-approximations.

Lastly, the empirical ratios obtained for branch-and-bound differ from the expected value of 1. Since the algorithm should return an optimal solution, this suggests either an implementation issue or an error in the experimental evaluation.

4 Conclusion

The greedy and LP-relaxation algorithms both achieved the expected worst-case approximation ratio of 2 on the tested values of n . Branch-and-Bound produced exact solutions but exhibited limited scalability. Further work is needed to understand why the empirical ratios obtained for Branch-and-Bound differ from the expected value of 1.